Week 8 Lab – What Are You Doing?

Objective: Refactor your Week 5 Inventory Management System using Object-Oriented Programming (OOP).

1. Replace product dictionaries with **Product** objects.
2. Create an **Inventory** class to manage the collection of products.
3. Apply **encapsulation** and the **Single Responsibility Principle (SRP)**.
4. Maintain the existing functionality:
    1. Display all products
    2. Add a product
    3. Update stock
    4. Search for a product
    5. Save inventory
    6. Exit

Key message: Same functionality, improved program structure.



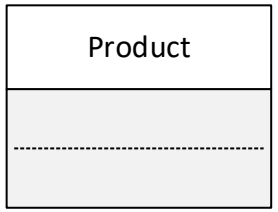
Quick Recap – Class and Object

Class

A blueprint defining attributes (data) and methods (behaviour).

Example:

- Vehicle, Car,
- Human, Employee
- Product



Object

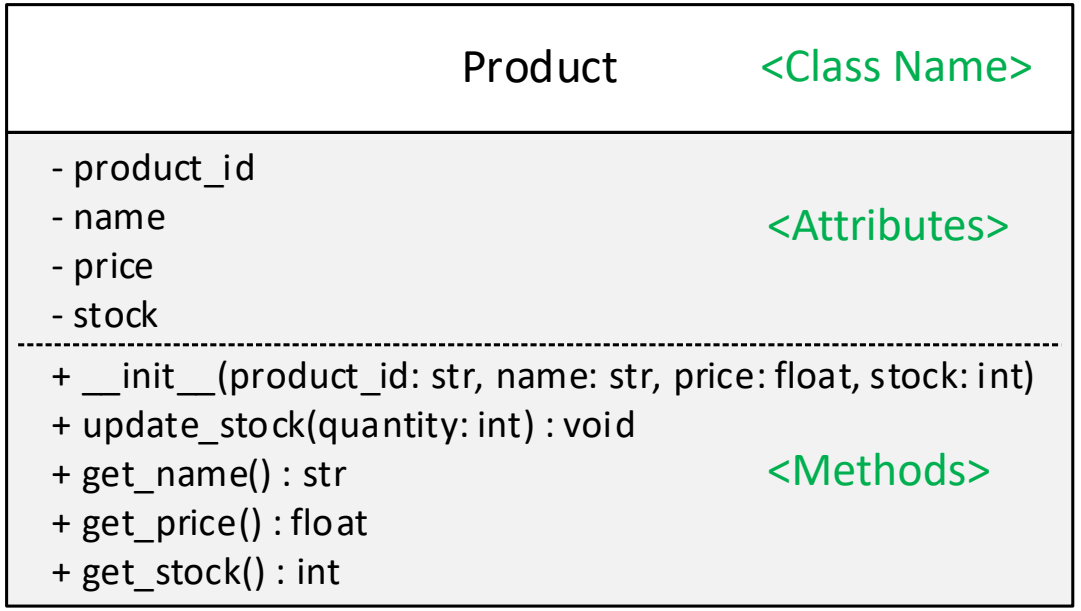
An instance of a class, with its own state.

Example:

- Toyota Corolla, Honda Civic,
- John, Mary
- Laptop



UML Class Diagram



UML vs Python notation

Visibility	UML	Python
Public	`+ name: str`	`self.name`
Protected	`# name: str`	`self._name`
Private	`- name: str`	`self.__name`

UML vs Python Methods

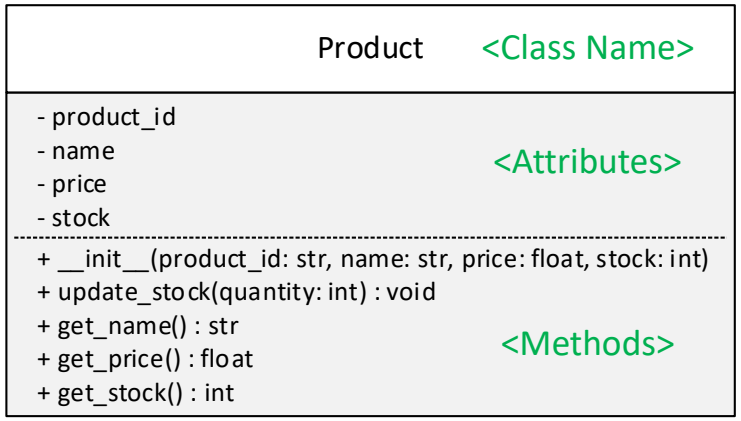
Visibility	UML Notation	Python Method
Public	`+ update_stock()`	`def update_stock(self):`
Protected	`# validate_stock()`	`def _validate_stock(self):`
Private	`- calculate_cost()`	`def __calculate_cost(self):`

*UML stands for **Unified Modeling Language**. It is a standardized general-purpose, object-oriented visual modeling language used to visualize, design, specify, and document the architecture and behavior of software systems (as well as business processes).*

Quick Recap – Example

```
INF1103_LabW8/
│
├── main.py
└── product.py
```

UML Class Diagram



product.py

```python
class Product:
    def __init__(self, product_id, name, price, stock):
        self.__product_id = product_id
        self.__name = name
        self.__price = price
        self.__stock = stock

    def update_stock(self, quantity):
        if quantity >= 0:
            self.__stock = quantity

    def get_name(self):
        return self.__name

    def get_price(self):
        return self.__price

    def get_stock(self):
        return self.__stock
```

main.py

```python
from product import Product

def main():
    # Create a Product object
    product1 = Product("P001", "Laptop", 1200.00, 15)

    # Call Product methods
    product1.update_stock(20)

    print(product1.get_stock())

if __name__ == "__main__":
    main()
```

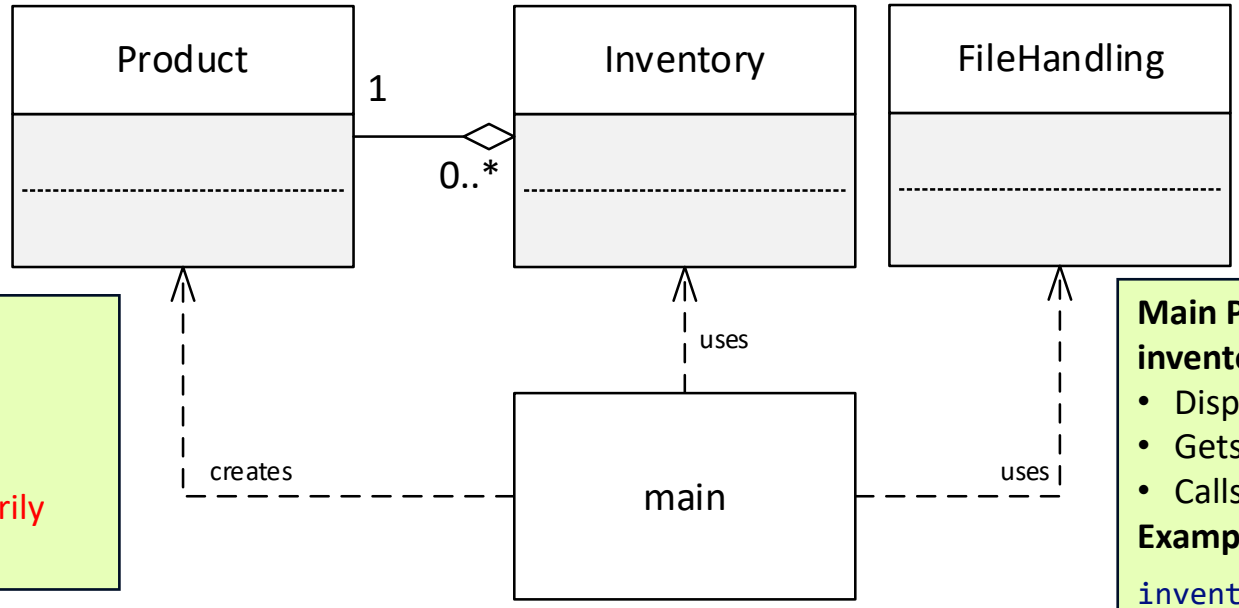
Key message:
A class defines the structure; objects represent individual products.

Quick reminders:

- **`__init__`** initializes an object's attributes.
- **`self`** refers to the current object.
- Methods define the operations an object can perform.
- **`__attribute`** indicates intended private access; use public methods to control changes.

SRP – Responsibilities for Week 8 Lab

Product
- Stores product information
- Manages its own state



File Handling
- Loads inventory data
- Saves inventory data

Implementation Options:
- If **FileHandling** converts **Product** objects into **JSON**, you may also introduce a dependency from **FileHandling** to **Product**.
- If **FileHandling** works only with ordinary **dictionaries** and **lists** provided by Main, no direct class relationship with **Product** is needed.

Inventory
- Stores products
- Adds products
- Searches products
- Displays products (Temporarily doing UI stuffs, Not ideal)

Main Program (for now also creates the products and inventory)
- Displays the menu
- Gets user input
- Calls the appropriate Inventory methods

Example:

```
inventory = Inventory()
product1 = Product("P001", "Laptop", 1200.00, 15)
inventory.add_product(product1)
```

Key message:
Put each operation where its responsibility belongs, not wherever it is easiest to write.

Classes	Relationship	Explanation
main → Inventory	Dependency	main creates or uses an Inventory object.
main → FileHandling	Dependency	main calls methods to load and save inventory data.
main → Product	Dependency	main creates Product objects.
Inventory → Product	Association (one-to-many)	Inventory maintains a collection of Product objects.

Design Check – Before You Continue

Self-check:

- ✓ Products are represented using **Product** objects, not dictionaries.
- ✓ **Inventory** manages a collection of **Product** objects.
- ✓ **Product** manages its own attributes and stock updates.
- ✓ The main program handles menus and user input.
- ✓ Private attributes are accessed or modified through appropriate public methods.
- ✓ Each class has **one clear responsibility** (SRP).
- ✓ UML class diagrams match the implemented classes.
- ✓ Existing functionality and JSON persistence still work.

Key message:

A working program is not enough. Your code should demonstrate good OOP design.